

Guia de ejercicios Arboles en C

Código base

Los siguientes ejercicios se basan en un arbol de busqueda. En este tipo de arbol, los valores menores se insertan hacia la izquierda y los valores mayores o iguales hacia la derecha.

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *leftChild;
    struct node *rightChild;
};

struct node *root = NULL;

void insert(int data){
    struct node *tempNode = (struct node*) malloc(sizeof(struct node));
    struct node *current;
    struct node *parent;

    tempNode->data = data;
    tempNode->leftChild = NULL;
    tempNode->rightChild = NULL;

    if(root == NULL) {
        root = tempNode;
    } else {
        current = root;
        parent = NULL;

        while(1) {
            parent = current;

            if(data < parent->data) {
                current = current->leftChild;

                if(current == NULL) {
                    parent->leftChild = tempNode;
                    return;
                }
            } else {
                current = current->rightChild;
```

```
        if(current == NULL) {
            parent->rightChild = tempNode;
            return;
        }
    }
}

void pre_order_traversal(struct node* root){
    if(root != NULL) {
        printf("%d ", root->data);
        pre_order_traversal(root->leftChild);
        pre_order_traversal(root->rightChild);
    }
}
```

Importante

Recorrido preorder: raiz -> subarbol izquierdo -> subarbol derecho.

2. Ejercicio 1: recorrido preorder basico

Dado el siguiente arreglo, inserte los valores en el arbol usando la funcion insert.

```
int array[7] = { 27, 14, 35, 10, 19, 31, 42 };
```

Preguntas:

- Dibuje el arbol que se forma.
- Indique cual es la raiz del arbol.
- Escriba el resultado del recorrido preorder.

Arbol esperado:

```

      27
     /  \
    14   35
   /  \  /  \
  10  19 31 42

```

Salida esperada:

```
Preorder traversal: 27 14 10 19 35 31 42
```

3. Ejercicio 2: cambiar los datos del arbol

Modifique el arreglo del programa principal por el siguiente y analice el arbol resultante.

```
int array[7] = { 50, 30, 70, 20, 40, 60, 80 };
```

Preguntas:

- Dibuje el arbol resultante.

- Muestre el recorrido preorder.
- Que nodos son hijos de 30?
- Que nodos son hijos de 70?

Arbol esperado:

```

      50
     /  \
    30   70
   / \  / \
  20 40 60 80

```

Salida esperada:

```
Preorder traversal: 50 30 20 40 70 60 80
```

4. Ejercicio 3: insertar valores desordenados

Use el siguiente arreglo de ocho elementos para construir un arbol binario de busqueda.

```
int array[8] = { 40, 25, 60, 15, 30, 50, 70, 65 };
```

Preguntas:

- Construya el arbol binario de busqueda.
- Indique el recorrido preorder.
- Cual es el hijo derecho de 60?
- Donde queda ubicado el nodo 65?

Arbol esperado:

```

      40
     /  \
    25   60
   / \  / \
  15 30 50 70
       /
      65

```

Salida esperada:

```
Preorder traversal: 40 25 15 30 60 50 70 65
```

5. Ejercicio 4: agregar recorrido inorder

Actualmente el programa solo muestra el recorrido preorder. Agregue una funcion para mostrar el recorrido inorder.

```
void in_order_traversal(struct node* root){
    if(root != NULL) {
        in_order_traversal(root->leftChild);
        printf("%d ", root->data);
        in_order_traversal(root->rightChild);
    }
}
```

```
}
```

Preguntas:

- Agregue la funcion `in_order_traversal` al programa.
- Llame a la funcion desde el `main`.
- Explique por que el recorrido inorder entrega los datos ordenados en un arbol binario de busqueda.

Salida esperada:

```
Preorder traversal: 27 14 10 19 35 31 42
Inorder traversal: 10 14 19 27 31 35 42
```

6. Ejercicio 5: agregar recorrido postorder

Agregue una funcion que recorra el arbol en postorder. Este recorrido visita primero la izquierda, luego la derecha y finalmente la raiz.

```
void post_order_traversal(struct node* root){
    if(root != NULL) {
        post_order_traversal(root->leftChild);
        post_order_traversal(root->rightChild);
        printf("%d ", root->data);
    }
}
```

Preguntas:

- Agregue la funcion `post_order_traversal` al programa.
- Llame a la funcion desde el `main`.
- Compare el resultado con preorder e inorder.

Salida esperada:

```
Postorder traversal: 10 19 14 31 42 35 27
```

7. Ejercicio 6: buscar un nodo en el arbol

Agregue una funcion que permita buscar un numero dentro del arbol binario de busqueda.

```
struct node* search(int data){
    struct node *current = root;

    while(current != NULL) {
        if(current->data == data) {
            return current;
        } else if(data < current->data) {
            current = current->leftChild;
        } else {
            current = current->rightChild;
        }
    }
}
```

```
    return NULL;
}
```

Preguntas:

- Pruebe la búsqueda con el número 31.
- Pruebe la búsqueda con el número 100.
- Explique por qué se compara si el dato buscado es menor o mayor que el nodo actual.

Salida esperada:

```
El numero 31 existe en el arbol
```

8. Ejercicio 7: contar los nodos del arbol

Cree una función recursiva que cuente cuantos nodos tiene el árbol.

```
int count_nodes(struct node* root){
    if(root == NULL) {
        return 0;
    }

    return 1 + count_nodes(root->leftChild) + count_nodes(root->rightChild);
}
```

Preguntas:

- Agregue la función count_nodes al programa.
- Muestre la cantidad total de nodos usando printf.
- Explique qué representa el valor 1 en la línea del return.

Salida esperada:

```
Cantidad de nodos: 7
```

9. Ejercicio 8: encontrar el valor menor

En un árbol binario de búsqueda, el valor menor se encuentra avanzando siempre hacia el hijo izquierdo.

```
int find_min(){
    struct node *current = root;

    if(current == NULL) {
        printf("El arbol esta vacio");
        return -1;
    }

    while(current->leftChild != NULL) {
        current = current->leftChild;
    }

    return current->data;
}
```

Preguntas:

- Agregue la funcion find_min al programa.
- Muestre el valor menor del arbol.
- Explique que ocurre si el arbol esta vacio.

Salida esperada:

```
Valor menor: 10
```

10. Ejercicio 9: encontrar el valor mayor

En un arbol binario de busqueda, el valor mayor se encuentra avanzando siempre hacia el hijo derecho.

```
int find_max(){
    struct node *current = root;

    if(current == NULL) {
        printf("El arbol esta vacio");
        return -1;
    }

    while(current->rightChild != NULL) {
        current = current->rightChild;
    }

    return current->data;
}
```

Preguntas:

- Agregue la funcion find_max al programa.
- Muestre el valor mayor del arbol.
- Compare la logica de find_max con find_min.

Salida esperada:

```
Valor mayor: 42
```

11. Ejercicio 10: contar hojas del arbol

Una hoja es un nodo que no tiene hijo izquierdo ni hijo derecho.

```
int count_leaf_nodes(struct node* root){
    if(root == NULL) {
        return 0;
    }

    if(root->leftChild == NULL && root->rightChild == NULL) {
        return 1;
    }
}
```

```
return count_leaf_nodes(root->leftChild) + count_leaf_nodes(root->rightChild);
}
```

Preguntas:

- Agregue la funcion count_leaf_nodes al programa.
- Muestre la cantidad de hojas.
- Escriba cuales son las hojas del arbol del ejercicio 1.

Arbol esperado:

```

      27
     /  \
    14   35
   / \  / \
  10 19 31 42
```

Salida esperada:

```
Cantidad de hojas: 4
```

12. Ejercicio final integrador

Modifique el programa original para que inserte los datos del siguiente arreglo y muestre informacion completa del arbol.

```
int array[9] = { 45, 20, 60, 10, 30, 50, 70, 25, 35 };
```

Preguntas:

- Muestre el recorrido preorder.
- Muestre el recorrido inorder.
- Muestre el recorrido postorder.
- Busque el numero 25.
- Muestre la cantidad total de nodos.
- Muestre el valor menor.
- Muestre el valor mayor.
- Muestre la cantidad de hojas.

Arbol esperado:

```

      45
     /  \
    20   60
   / \  / \
  10 30 50 70
   / \
  25 35
```

Salida esperada:

```
Preorder traversal: 45 20 10 30 25 35 60 50 70
Inorder traversal: 10 20 25 30 35 45 50 60 70
Postorder traversal: 10 25 35 30 20 50 70 60 45
El numero 25 existe en el arbol
```

Cantidad de nodos: 9

Valor menor: 10

Valor mayor: 70

Cantidad de hojas: 5

13. Resumen de recorridos

Complete o revise los resultados de los principales recorridos trabajados en la guía.

Ejercicio	Arreglo	Recorrido solicitado	Resultado esperado
1	{27,14,35,10,19,31,42}	Preorder	27 14 10 19 35 31 42
2	{50,30,70,20,40,60,80}	Preorder	50 30 20 40 70 60 80
3	{40,25,60,15,30,50,70,65}	Preorder	40 25 15 30 60 50 70 65
4	{27,14,35,10,19,31,42}	Inorder	10 14 19 27 31 35 42
5	{27,14,35,10,19,31,42}	Postorder	10 19 14 31 42 35 27
Final	{45,20,60,10,30,50,70,25,35}	Pre/In/Post	Ver salidas esperadas